Design Document: AI-Native Compliance Infrastructure

> **Disclaimer:** Independent personal project for self-learning. Not affiliated with any employer, state agency or vendor. Uses only public documents and synthetic data. No internal or confidential data and no personal data (no SSNs, member or patient records, PHI/PII). Views are my own. See DISCLAIMER.md.

Author: Shubham Singh | Status: working prototype, evaluated, known weaknesses documented | Last updated: October 2026

This is the single document covering the project from Phase 0 (setup) to the latest evaluation run: goals, architecture, every phase, key decisions, results, failures, and what is next. Related files: `README.md` (summary), `eval/README.md` (evaluation details and run log), `docs/LESSONS_LEARNED.md` (problems and fixes), `docs/SETUP_GUIDE.md` (setup).

---

0. Summary

The project has two versions. The original language-model pipeline found every change (recall 1.0) but raised false alarms, kept requirements the policy had removed, and gave different answers on identical input: 6 of 10 test cases passed. Those are problems of asking a model to do exact work, so the checkable work moved to a **rule-based engine** (Phase 9). It passes 10 of 10 development cases and 8 of 8 blind cases, the same way every run, including the real MITA 2.0 to 3.0 pair, and 96.7% to 98.5% of checks on 200 generated policy manuals.

The engine now runs inside a **compliance agent** (Phase 10): it scans folders of policies and SOPs, writes suggested changes as diffs, never edits a file without a named approver, logs every action, and is usable from any MCP-capable assistant. A narrow language-model check handles the one thing rules cannot (meaning without shared words); it is built but its accuracy is not yet measured (Phase 11).

How far to trust this: the rules were tuned on the development cases, the generated tests use my own templates, and only one real document pair has been tested. Whether anyone needs this product is **not yet validated**.

---

1. Project overview

1.1 The problem in plain words

Organizations that run under government health programs (Medicaid is the example used here) work from two kinds of documents:

- **Policy documents** written by someone else (a federal or state agency): rules, bulletins, certification checklists, framework documents.
- **SOPs** (standard operating procedures) written inside the organization: step-by-step instructions that tell staff how to follow those rules, for example “request prior authorization before visit 13”.

Policies change. When one does, a person must (1) find what changed, (2) work out which SOPs mention or depend on it, (3) rewrite them, and (4) prove later, in an audit, that the SOPs matched the policy at the time. Done by hand this is slow, repetitive and easy to get wrong. A missed SOP stays out of date silently, and the mistake surfaces later as an audit finding.

1.2 The idea

Let software do the first pass: read the old and new policy, list what changed, name the SOPs affected, draft the new SOP wording with a citation to the policy text, and mark anything uncertain for a person. The person still decides; the tool removes the reading and searching.

1.3 Why this project exists

This is a self-learning and portfolio project, built independently. It teaches how to build a retrieval-augmented generation (RAG) system on free hardware, how to measure whether an AI system is actually right, and whether this problem is worth turning into a product. It is **not** an employer project and uses no employer or agency data.

---

2. Goal, objectives, scope

2.1 Goal (one sentence)

Show, with measured evidence, whether an AI pipeline built only from public documents and free hardware can reliably turn a policy change into correctly scoped, reviewable SOP updates.

2.2 Objectives

#	Objective	How it is measured	Status
O1	Build a working end-to-end pipeline (ingest, retrieve, detect, map, draft, flag) on free hardware	Pipeline runs start to finish on a Kaggle T4	Done
O2	Find the real changes between two versions	Change recall (proposed target 0.9 or higher)	Language model: 1.0 on short cases, missed the real long document. Rule engine: 1.0 on all cases, including the real MITA pair
O3	Flag only the SOPs that are truly affected	Impact precision and recall (proposed target 0.9 or higher each); zero false alarms on a wording-only change	Language model: recall 1.0, precision 0.889, wording-only control fails. Rule engine: 1.0 and 1.0, control passes
O4	Edit SOPs correctly: change what must change, keep everything else, delete what the policy removed, invent nothing	Required and forbidden text per SOP; new-clause flags	Language model: numbers work, removed requirements fail. Rule engine: both pass; added requirements are copied word for word, never invented
O5	Keep a human in control	Every draft carries review flags; nothing is “approved” automatically	Done
O6	Measure honestly and publish the failures	Answer keys written before runs, holdout cases, repeated runs, lessons log	Done, ongoing
O7	Decide whether this is worth building further	Conversations with real practitioners	One conversation so far; the biggest open risk

The targets are my own proposed bars for “good enough to be useful as a draft tool”. They are not an industry standard.

2.3 Scope

In scope

- Public policy documents (MITA framework documents, CMS-style bulletins) and invented SOPs.
- Comparing two versions of one document; mapping the differences to a set of SOPs; drafting SOP text; flagging for review.
- An evaluation harness with answer keys, a real document pair, and repeated runs.
- Demo pages, documentation, and a public GitHub record of the work, including failures.

Out of scope (on purpose)

- Any employer, agency, member, provider or patient data. No SSNs, PHI or PII anywhere.
- Real internal SOPs. All SOPs are invented.
- Legal or regulatory advice, or any claim that a draft is compliant.
- Automatic approval or publishing of SOP changes.
- A production system, login, security hardening, or multi-user hosting.
- Paying for compute. Everything must run on the free Kaggle tier.

2.4 Who it is for (hypothetical users)

Role	What they would get
Policy or compliance analyst	A list of what changed and where, instead of reading both versions line by line
SOP owner	A shortlist of the SOPs that may need an update, with the reason
Reviewer or auditor	A draft with a citation to the policy text and flags on anything new, plus a trail of what was changed and why

These users are assumptions. Nobody outside this project has yet confirmed they would use it.

---

3. Impact: what it is meant to change

The intended impact, and an honest note on what is proven:

Intended impact	How the design supports it	Proven so far?
Faster first pass on a policy change	The tool reads both versions and proposes a change list and drafts	Not measured in minutes. Only accuracy on test cases is measured
Fewer missed SOPs	Every SOP is checked against every change	Recall is 1.0 on the test cases; the test set is small
Fewer wrong edits	Only the changed parts are edited; invented sentences are flagged	Works for numbers; fails for removed requirements
Audit traceability	Each draft cites the policy text it came from	Citations work in the pipeline; no audit has used them
Trust through honesty	False alarms and misses are counted and published	Yes: see the lessons log
Learning value for the builder	Full RAG build, evaluation design, hardware limits, failure analysis	Yes

The biggest impact risk is a **noisy tool**: if it raises false alarms, reviewers stop trusting it. This is why precision and the wording-only control matter as much as recall.

---

4. Constraints and data rules

Constraint	Consequence
Free compute only (Kaggle T4 x2, about 14.5 GiB of GPU memory each, about 30 GPU hours per week, sessions reset after 9 hours)	4-bit 7B model, small prompts, section-by-section comparison for long documents
Public documents and synthetic data only	Internal SOPs are invented. Real documents are public MITA/CMS materials
Personal project, no employer ties	Disclaimer in every notebook, demo page, README and this document
Shell cannot reach medicaid.gov	Real PDFs were uploaded manually or fetched inside Kaggle

---

5. Architecture

```
old version + new version (public policy text)
        |
  [1 Ingest]       split into chunks, embed (MiniLM), index (FAISS)
        |
  [2 Detect]       LLM lists substantive changes (whole doc, or section by section if long)
        |
  [3 Map impact]   per SOP: affected or not, structured JSON with a reason
        |
  [4 Draft]        rewrite only what the change requires, cite the policy
        |
  [5 Flag]         any new sentence not close to the original SOP is marked for human review
        |
  [6 Report]       JSON output, redline demo page, dashboard
```

This is the original language-model pipeline (Phases 2 to 8). The current version replaces steps 2 to 5 with the rule-based engine and adds a narrow language-model check for meaning; see Phases 9 to 11.

Models: Qwen2.5-7B-Instruct (4-bit, bitsandbytes) for generation; sentence-transformers all-MiniLM-L6-v2 for embeddings; FAISS for search.

Data shapes

- Document: `{id, title, text, metadata:{policy_family, version, effective_date, is_current}}`
- SOP: `{id, title, text}`
- Pipeline result: `changes_summary, impact_analysis, impact_parse_method, affected_sop_ids, drafted_updates[{sop_id, sop_title,`

`original_text, draft_text, review_flags}]`

---

6. Phases, from the start to now

Each phase below says **what** was done, **why**, **how**, what was **seen when it ran**, and its **impact** on later phases. Phases 0 to 2 are written in full detail because they set the foundation for everything after.

Phase 0 - Environment setup

What. Choose and set up the tools, accounts and models, and prove the GPU works.

Why. Every later decision (model size, prompt length, how long documents are handled) is limited by this hardware. Choosing the tools deliberately, and cheaply, was the first design decision.

How.

1. **Accounts (free):** Kaggle (notebooks plus a free GPU), GitHub (code storage), Hugging Face (to download the open model).
2. **Accelerator:** in Kaggle, Settings, Accelerator, GPU T4 x2. Internet switched on (needs phone verification) so the model can be downloaded.
3. **Libraries installed** (`pip install`), and why each:

Library	Role
`transformers`	Loads and runs the language model
`accelerate`	Places the model on the GPU automatically (`device_map="auto"`)
`bitsandbytes`	Shrinks the model to 4-bit numbers so it fits in GPU memory
`sentence-transformers`	Turns text into number vectors (embeddings)
`faiss-cpu`	Fast similarity search over those vectors
`pypdf`	Reads text out of PDF files

4. **GPU check:** the notebook prints whether a GPU exists. Result in the run: `GPU available: True, Tesla T4, 15.6 GB` (15.6 GB in decimal units is the same as the 14.56 GiB that later error messages show).

The two models, and why these.

	Embedding model	Generation model
Name	all-MiniLM-L6-v2	Qwen2.5-7B-Instruct
Job	Turn each text chunk and each question into a 384-number vector so similar meanings sit close together	Read retrieved text and write the answer
Size	About 90 MB, runs fast	About 15 GB of weights downloaded; about 5 GB in GPU memory once shrunk to 4-bit
Why chosen	Tiny, free, good enough for retrieval	Strong at following instructions for its size; small enough for a free T4; open, so no per-call cost and no data leaves the notebook

Alternatives considered.

- A hosted large model through an API: more accurate, but costs money per call and was outside the free-only constraint. Also less useful for learning how the pieces work.
- A smaller model (for example 3B): fits easily but follows structured instructions less reliably.
- A bigger model (13B or larger): does not fit the free GPU. Rented GPUs (RunPod, Colab Pro) remain an upgrade path.

4-bit quantization in one line. Normally each model weight is stored with 16 bits. 4-bit stores it with 4, so the model needs roughly a quarter of the memory, at a small cost in quality. Settings used: `load_in_4bit`, `bnb_4bit_compute_dtype=float16`, `bnb_4bit_use_double_quant=True`.

Seen when it ran. The model download was 15.2 GB (about 70 seconds on Kaggle), then loaded into the GPU. The notebook text said “about 5 GB download”; that was wrong: the download is 15 GB and the 5 GB figure is the in-memory size after shrinking.

Impact. The hardware sets a ceiling. Later, this ceiling is exactly what blocked whole-document comparison of the real MITA pair (a 10,500-word prompt asked for 26 GiB of GPU memory). The fix was software (comparing section by section), not more hardware. Phase 0 is therefore where "free tier only" became a design rule.

Phase 1 - Data collection (public and synthetic only)

What. Create the documents the pipeline will read.

Why. The project needs documents where the right answer is known, so that the AI can be graded. It also needs a strict rule about what data is allowed, set before any building starts.

How.

- A list called `SAMPLE_DOCS`. Each entry is a small dictionary: `id`, `title`, `text`. Three documents were written in a public-policy style (invented, not copied from any agency):
    1. **Prior-authorization policy, version 1:** authorization needed after 12 outpatient physical therapy visits per year; request at least 5 business days before visit 13.
    2. **Prior-authorization policy, version 2 (updated):** threshold raised to 20 visits; request at least 3 business days before visit 21.
    3. **Telehealth reimbursement policy:** parity for behavioral health only; audio-only visits paid at 80% of the standard rate; modality must be documented on each claim.
- A helper `load_pdfs_from_folder()` using `pypdf`, ready for real public PDFs. It stayed unused until the real MITA documents were added in Phase 7.
- Planned real public sources: medicaid.gov state plan amendments, CMS provider bulletins, Federal Register Medicaid rules.

Why synthetic text first. The change between version 1 and version 2 was planted on purpose (two numbers). That gives a known "right answer" to test against, and avoids any question about whose data it is.

Why the data rule matters. No employer, agency, member or patient data, and no SSNs, PHI or PII, ever. Fixed from day one, repeated in a disclaimer on every notebook, page and README, because an early lapse would be hard to undo.

Limitations seen.

- The version information lived only inside the free-text title ("v1", "v2 - UPDATED"). Nothing machine-readable said which version is current. This turned out to matter in Phase 2.
- Three short documents are a toy corpus. Real documents are dozens of pages.

Impact. The two prior-authorization versions became the template for the later test cases. The title-only versioning gap led directly to the explicit version metadata added in Phase 3.

Phase 2 - Core RAG engine

What. Build a question-answering engine that reads the documents and answers from them, citing sources.

Why. RAG ("retrieval-augmented generation") is the base of the project. A language model alone can invent facts. RAG first finds the relevant source text, then makes the model answer only from it, so answers can be checked. Phases 3 and 4 reuse these pieces.

How, step by step.

1. **Chunking** (`chunk_text`, 300 words per chunk, 50 words overlap). Long text is cut into pieces so search can find the right passage rather than a whole document. The overlap stops a sentence from being cut in half and losing context at a boundary. *Seen:* the three sample documents each have under 300 words, so each became exactly one chunk (3 chunks). The chunking logic was therefore **not actually exercised** until real long documents arrived. Later, section comparison used smaller chunks (about 180 words) because change detection needs finer pieces than question answering.
2. **Embedding.** Each chunk is converted into a vector of 384 numbers by MiniLM. Texts with similar meaning get nearby vectors.
3. **Index.** The vectors go into a FAISS `IndexFlatL2` index. "Flat" means exact search by distance to every vector: simple, no tuning, fast enough for a small corpus. (Approximate indexes only matter at much larger sizes.)
4. **Retrieval** (`retrieve(query, top_k=3)`). The question is embedded the same way and the closest chunks are returned. *Seen:* for "What is the prior authorization threshold for physical therapy?", the ranking was sensible (version 2 first, version 1 second, telehealth last). But `top_k=3` on a 3-chunk corpus returns everything, so this was a weak test of retrieval quality.
5. **Generation** (`ask(query, top_k=3, max_new_tokens=300)`). The retrieved chunks are placed in a prompt with an instruction to answer using only that context and to cite the source; Qwen2.5-7B writes the answer. Limiting the answer to the retrieved text is what keeps it grounded.

What happened when it ran (the most important result of Phase 2).

Question	Answer	Verdict
“What is the prior authorization threshold for physical therapy?”	“12 visits per calendar year”, sourced to version 1	**Wrong.** Version 2 (20 visits) had been retrieved too, but the model used the old version. The engine has no idea which version is current.
“What changed about telehealth reimbursement?”	“The context does not mention any changes”	Honest and correct (only one version exists), but it shows the engine has no concept of “a change” at all.

Why this matters. Plain RAG can look up facts but cannot tell which version is current and cannot compare versions. That gap is the product opportunity, and it justifies everything after Phase 2: version tags and a change-detection step (Phase 3), mapping changes to SOPs and drafting updates (Phase 4).

Decisions in Phase 2.

Decision	Reason	Cost
Ground answers in retrieved text only	Reduces invented facts, allows citation	Answers are only as good as the retrieval
Exact (flat) index	Simple and correct for small data	Would need change at large scale
Small embedder on the same GPU	Fast, free	Less precise than larger embedders
Word-based chunking with overlap	Easy to understand and tune	Ignores headings and paragraph structure

Impact. Phase 2 delivered the working base (ingest, embed, index, retrieve, generate) and, more usefully, proved its own limitation. The “wrong version” answer is the clearest justification for the rest of the project. Open improvements, not yet done: use version metadata to filter retrieval to the current version, chunk by document headings, and test retrieval on a larger corpus.

Phase 3 - Change-detection (diff) layer

- Tag each document with version metadata (family, version, effective date, current flag).
- Detect changes: send old and new text to the LLM and ask for substantive changes.
- Map changes to SOPs: the model returns structured JSON per SOP (affected yes or no, reason).
- Generate a full report. Notebook: `notebooks/phase3_diff_layer.ipynb`.

Problem found: an unaffected SOP (telehealth) was flagged because the model wrote a sentence saying it was not affected and string matching read it as a hit. **Fix:** structured JSON output with a negation-aware fallback parser (`patches/sop003_fix_cells.py`).

Phase 4 - Draft generation

For each affected SOP, draft updated text that changes only what the policy changed, with a citation. One function, `run_full_compliance_check(old_doc, new_doc, sops)`, runs everything. Notebook: `notebooks/phase4_draft_generation.ipynb`.

Problem found: the model sometimes invented sentences the policy never asked for. **Fix:** a flagger (`new_clause_flags`, similarity threshold 0.8) marks any sentence in a draft that does not closely match the original SOP, so a reviewer sees it (`patches/draft_prompt_fix_cells.py`).

First result on the sample change (prior-authorization threshold 15 to 20 visits): 2 of 3 SOPs flagged correctly, the telehealth SOP correctly left alone.

Phase 5 - Demo and export

Results exported to JSON (`data/compliance_results.json`). Two static pages: `demo/compliance_demo.html` (redline view of old vs drafted SOP text with review flags) and `demo/compliance_dashboard.html`. Both carry the disclaimer banner.

Phase 6 - GitHub packaging and documentation

Repo reorganized (notebooks, patches, demo, data, docs, eval). Swim-lane diagram of the pipeline, rewritten README, project overview PDF with product recommendations, setup guide, DISCLAIMER.md.

Phase 7 - Evaluation harness

Motivation: one hand-picked example proves little. Question: does the AI find and correctly apply real changes?

Design (in `eval/`):

- **Cases** in `eval/cases/`, each with old text, new text, SOPs (some that must not change), and an **answer key written before the AI runs**: expected changes (with key terms), affected SOP ids, required and forbidden text in edited SOPs, or "expect no change".
- **Categories:** certification (checklist and deadline changes), MITA-style, general policy (threshold change, requirement added, requirement removed), and a wording-only control.
- **Holdouts:** cases the prompts were never tuned on (`cert_02`, `policy_04`, `policy_05`) to expose overfitting.
- **Scorer** (`score.py`, no GPU): change recall, SOP precision and recall, edit correctness, drafts for unaffected SOPs, new-clause flags. Self-tested with fake pipelines (perfect, do-nothing, flag-everything) in `test_harness.py`.
- **Repeats:** each synthetic case runs 3 times because the model samples; reports always-pass, flaky and never-pass.
- **Real case:** MITA introduction (FM6), version 2.0 (2006) vs 3.0 (2012), public documents. The 2.0 to 3.0 change of interest is eight business areas becoming ten. Matching synthetic SOPs: a business-area inventory (must change) and a maturity-scoring SOP (must not be flagged).

Phase 8 - Making long documents fit (section mode)

Problem: the whole MITA pair is about 10,500 words and asked the GPU for 26 GiB. This is a design limit, not a hardware shortage.
Approach (`eval/sectioned.py`):

1. Split both documents into chunks of about 180 words.
2. Match each new chunk to its top 2 old chunks (TF-IDF cosine), non-exclusively.
3. Drop identical pairs and wording-only pairs; keep pairs whose differences involve numbers, number words or obligation words (ignoring version labels, years, months).
4. Run change detection per pair, keep a finding only if it mentions a really-changed term, drop duplicates, and cap the merged summary at about 3,500 characters.
5. Documents under about 1,500 words skip all this and are compared whole.

The real MITA pair produces 68 candidate chunk pairs, and the real 8 to 10 pair is among them (checked locally with a simulated model).

Phase 9 - Rule-based engine (decision: move checkable work out of the model)

Why. Repeated runs showed the same weaknesses: false alarms on wording-only changes (0/3), removed requirements kept in drafts (0/3), real MITA change missed, and different answers on identical input. These are not hardware problems; they come from asking a language model to do exact work. Buying a bigger GPU would not make a model deterministic.

What. `eval/engine.py`, no model and no GPU. It decides changes, affected SOPs and drafts by rules (details and limits in `eval/README.md`): sentence alignment with wording-only filtering, numbers and dates compared as facts, counted things compared across the whole document, removal notices, header and footer cleaning, copy-only (never invented) text for added requirements, and a VERIFY flag on every edit.

Result. 10 of 10 development cases and 8 of 8 blind cases pass, identical across repeated runs, including the real MITA 8 to 10 change and the maturity-scoring SOP left alone.

Scale test. A generator (`eval/generate_cases.py`) builds labeled cases with exact answers; 4,000 cases across four seeds pass at 97.6 to 98.3%. All failures are one category: an added requirement whose SOP shares no words with it, which needs a language-model check.

How far to trust it. The rules were tuned on the development cases. The blind cases found one real gap on the first run (month names) which was then fixed. Rules cannot judge meaning, so a requirement added in words the SOP never uses is missed. It has never seen a real SOP or a second real document pair. "No errors" is not a claim anyone can honestly make for any AI system; the design goal is that errors are rare, visible and caught by a person.

Next. A hybrid: rules decide everything checkable, and a language model of any size (hardware is no longer the limit) answers only narrow yes/no questions about uncertain pairs. More real document pairs and more fresh blind cases.

Phase 10 - Compliance agent and MCP connector

Goal. Move from "an engine that passes test cases" to "an agent that scans documents and suggests changes", usable from any MCP-capable assistant.

Design. `compliance_agent/` has four parts: `store.py` (reads txt, md, pdf, docx from folders and orders versions by date or version number; this is the one place new sources such as Drive, SharePoint or a git repo would plug in), `engine.py` (the rule engine), `scan.py` (compare each new version with the previous one, write the report, drafts, patches and audit log, skip pairs already done), and `mcp_server.py` (seven tools).

Safety rules built in. A scan never edits a source file (tested byte for byte). Suggestions go to an output folder as diffs. `apply_draft` needs a named approver, keeps a backup and writes an audit line. Anything the rules cannot classify is listed as "needs a person" instead of being guessed.

Tested. `tests/test_agent.py` runs the full folder workflow on `demo_workspace/`, checks exact draft text, rescans, approval and the audit log, and talks to the real MCP server over stdio.

Found by running it for real. A removal edit that left "mail a" in the text, a report overwritten on rescan, and a misleading flag. Each is fixed and covered by a test.

Next. Language-model check for the one failure type rules cannot handle (requirements worded with none of the SOP's words), more sources (Drive, SharePoint, git, web pages with change monitoring), a review screen, and real document pairs at larger scale.

Phase 11 - Realistic manuals and the language-model check

Realistic manuals. `eval/generate_documents.py` builds whole synthetic manuals (sections, bullets, page headers) with their own SOP libraries and scores the real folder scan. It exposed that the tiny-case score (98%) overstated the agent: on manuals it first scored 74%. The fix was sentence splitting (headings and bullets no longer glued to the next sentence) plus a faster, indexed SOP matcher; it now scores 96.7% to 98.5% across seeds, with every real change found and edited correctly. The remaining misses are false alarms from one shared word, the case for the language-model check.

Language-model check (judge).

Rules decide what can be checked exactly; a judge handles meaning, in three narrow yes/no questions (requirement applies to an SOP with no shared words, weak one-word link real or not, reworded sentence same requirement or not). It never writes text, "unsure" changes nothing, and every answer is cached and logged. Backends: local open model (`hf`), hosted model (`api`), or none. Tested with fake judges for plumbing and safety; the real model's accuracy is measured by `eval/kaggle_run_judge.py` (rules only versus rules plus judge, per case type). Not yet measured.

---

7. Key design decisions and why

Decision	Why	Trade-off
Open 4-bit 7B model on a free GPU	Zero cost, fully reproducible	Weaker than large hosted models; noisy output
Structured JSON for impact mapping	String matching misread negations	Parse failures need a fallback
Human-review flag on new clauses	Model invents text; reviewer must see it	More review work
Answer keys before running the AI	Prevents grading to fit the output	Cases take effort to write
Holdout cases and repeated runs	Avoids tuning to the test and mistaking noise for signal	More GPU time
Section-by-section for long docs	Whole docs do not fit in memory	Misses changes that span chunks; hides added/removed sentences on short docs (hence the size routing)
Draft is always "for human review"	Compliance cost of a wrong silent edit is high	No full automation claim

---

8. Results

Current version: rule-based engine (no GPU, same answer every run). 10 of 10 development cases and 8 of 8 blind cases pass. 4,000 generated cases across four seeds: 97.6% to 98.3%. 200 generated whole manuals run through the real folder scan: 96.7% to 98.5% of checks. Trust limits are in Phase 9 and section 9.

The runs below are the original language-model pipeline. They are kept because they are the evidence for the move to rules.

Run 1 (original prompts, whole-document, 7 cases): 4 passed, 3 failed. Failures: requirements added or removed were not handled because the prompts forbade it, and SOPs that only mentioned the change indirectly were missed. Led to the v2 prompts (affected if the change alters something the SOP states, or adds or removes a requirement for the activity the SOP performs; draft may delete a removed requirement or add one short sentence).

Run 2 (v2 prompts, 10 cases, one run each): about 4 to 5 of 10, different cases failing in different runs. The disagreement on identical input

showed single runs are noisy, which led to repeated runs.

Run 3, whole-document, 3 repeats each: 6 of 10 passed. Change recall 1.0, impact precision 0.889, impact recall 1.0. Stable failures: `policy_05` removed requirement stays in the draft (0/3, a holdout, so the v2 prompt fitted only the case it was tuned on) and `control_01` false alarm on a wording-only change (0/3). `cert_01` flaky (2/3). The real MITA case ran out of memory, as expected.

Run 3, section mode, one run each: 6 of 10 passed, no crash. The capped summary fixed the out-of-memory error. But the short synthetic cases `policy_03`, `04`, `05` failed ("no draft produced") because section mode hid added and removed sentences; this led to routing short documents to whole-document comparison. On the real MITA case, 68 chunk pairs were checked and only 4 findings kept; the 8 to 10 change was missed and the maturity SOP was wrongly flagged. Whether the model or the filter is at fault is not yet known (kept findings are now printed for the next run).

---

9. Known weaknesses (open problems)

Current version (rule engine and agent):

1. **Rules cannot judge meaning.** A requirement added in words the SOP never uses is missed. This is the only failure type in the generated tests, and the reason for the language-model check, whose accuracy is not yet measured.
2. **Tuned on its own tests.** The rules were developed against the 10 development cases, and the blind set is partly used up after one fix. Fresh blind cases are needed for an unbiased number.
3. **Generated tests use my own templates.** They show robustness on this family of changes, not accuracy on real policy writing.
4. **One real document pair, no real SOPs.**

Original language-model pipeline (fixed by the rule engine, kept for the record):

1. **Removed requirements stay in drafts.** Failed in all 3 whole-document runs on a holdout.
2. **False alarms.** The wording-only control fails 0/3 in whole-document mode; the maturity SOP was flagged wrongly in both real-case runs. In compliance work, noise erodes trust.
3. **Real MITA 8 to 10 change missed in section mode.** Cause not yet identified.
4. **Model noise.** Even identical inputs vary run to run.
5. **Tiny, partly self-written test set.** 10 cases. Eight are synthetic and written by me, so results say little about real-world accuracy.
6. **Untested on real SOPs or real versioned CMS documents** beyond one pair.

---

10. Planned fixes (next)

Current:

- Measure the language-model check: rules only versus rules plus judge, per case type (`eval/kaggle_run_judge.py`).
- Write fresh blind cases and add a second real document pair with a hand-written answer key.
- Talk to practitioners before building more (objective O7).

Earlier plan for the language-model pipeline (superseded by the rule engine):

- **Removal handling:** ask the model only to name the sentence to delete; delete it in code; if the named text is not found verbatim in the SOP, route to human review. Deterministic and testable.
- **False alarms:** flag an SOP only when the changed terms from the detected change actually appear in it.
- **Real case:** read the printed `kept:` findings and the saved JSON to find whether the model or the filter lost the 8 to 10 change, then fix that piece only.
- **Do not change prompts again** until repeated runs show a stable failure.
- Update `eval/README.md` and this document with the new numbers after each run.

---

11. Roadmap

- ☒ Phases 0 to 5: ingest, retrieve, detect, map, draft, flag, demo
- ☒ GitHub packaging, disclaimer, swim-lane diagram, overview PDF
- ☒ Evaluation harness with answer keys, holdouts, repeats, one real case
- ☒ Section mode for long documents, size routing for short ones
- ☒ Rule-based engine fixes removal and false-alarm problems on all current cases (10/10 development, 8/8 blind)
- ☒ Compliance agent: folder scan, approval-gated edits, audit log, MCP server

- [x] Generated scale tests: 4,000 cases and 200 whole manuals
- [x] Hybrid: narrow language-model yes/no check for uncertain pairs (built and tested for safety)
- [ ] Measure the language-model check's accuracy
- [ ] Fresh blind cases and a second real document pair
- [ ] Real MITA case passing in section mode
- [x] 18 hand-written cases (9 synthetic, 1 real pair, 8 blind)
- [ ] More real old/new public document pairs
- [ ] Customer conversations (agency staff, vendors, practitioners in other states)
- [ ] Optional: thin reporting prototype (status per SOP, synthetic quarterly summary)
- [ ] Optional: hosted backend so results persist

---

12. Risks

Risk	Note
Market need unvalidated	A first practitioner conversation suggested the manual process may already be good enough. Customer interviews come before more building.
Accuracy	The rule engine passes every current test, but those tests are small, partly tuned on, and mostly synthetic. That does not support a claim of real-world reliability. Draft-only, human-in-the-loop is the only responsible framing.
Scope and ownership	This is a personal project built on public documents and synthetic SOPs. Check any employment agreement on side projects and IP before commercializing anything.
Public repo	The repo was made public for sharing. Set it back to private if you do not want it indexed; the link then stops working for others.

---

13. Repository map

```
README.md                summary, results, how to run
DISCLAIMER.md            scope and data rules
notebooks/               starter (phases 0-2), diff layer (3), draft generation (4)
patches/                 structured impact fix, draft prompt fix and flagger, prompt v2
demo/                    redline demo page, dashboard
data/                    compliance_results.json from a real run
docs/                    this document, lessons learned, setup guide, swim-lane, overview PDF
eval/                    harness: cases, scorer, section mode, mock pipelines, real case, run log
```

Independent personal project. No proprietary, client, employer or agency data, and no personal data of any kind, is used anywhere in this repository.